

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN

ĐỀ THI VÀ BÀI LÀM

Tên học phần: Toán ứng dụng CNTT

Mã học phần: 21Nh12

Hình thức thi: *Tự luận có giám sát*

Đề số: **0003**

Thời gian làm bài: 90 phút (*không kể thời gian chép/phát đề*)

Được sử dụng tài liệu khi làm bài.

Họ tên: Phan Trần Nhật Hạ

Lớp: 21TCLC_DT2

MSSV: 102210159

Sinh viên làm bài trực tiếp trên tệp này, lưu tệp với định dạng MSSV_HọTên.pdf và nộp bài thông qua MS Teams.

Câu 1 (2 điểm): Cho $M=50$ và $N=500$. Viết chương trình (có sử dụng hàm) thực hiện:

- a) (1.0 điểm) Tìm số lượng các số nguyên tố nằm trong khoảng M đến N , liệt kê và tính tổng của chúng.

Trả lời: Dán code bên dưới:

```
#include <iostream>
#include <math.h>
using namespace std;

bool checkPrime(int n)
{
    if(n<2) return false;
    for(int i=2; i<=sqrt(n); i++)
    {
        if(n%i==0) return false;
    }
    return true;
}
```

```

int main() {

    int M, N;

    cout << "Nhap gia tri M: ";

    cin >> M;

    cout << "Nhap gia tri N: ";

    cin >> N;


    cout << "Cac so nguyen to trong khoang tu " << M << " den " << N << " la: \n";

    int count = 0, sum = 0;

    for (int i = M; i <= N; i++)

        {

            if (checkPrime(i)) {

                count++;

                sum += i;

                cout << i << " ";

            }

        }

    cout << endl << "So luong cac so nguyen to trong khoang tu " << M << " den " << N << " la: " << count << endl;

    cout << "Tong cac so nguyen to trong khoang tu " << M << " den " << N << " la: " << sum << endl;

    return 0;

}

```

Trả lời: Dán kết quả thực thi vào bên dưới:

```

D:\LEARNING\DailyHoc\HK5\Tc x + ~
Nhap gia tri M: 50
Nhap gia tri N: 500
Cac so nguyen to trong khoang tu 50 den 500 la:
53 59 61 67 71 73 79 83 89 97 101 103 107 109 113 127 131 137 139 149 151 157 163 167 173 179 181 191 193 197 199 211 223 227 229 233 239 241 251 257 263 269 271 277 281 283 293 307 311 313 317 331 337 347 349 353 359 367 373 379 383 389 397 401 409 419 421 431 433 439 443 449 457 461 463 467 479 487 491 499
So luong cac so nguyen to trong khoang tu 50 den 500 la: 80
Tong cac so nguyen to trong khoang tu 50 den 500 la: 21208

-----
Process exited after 2.428 seconds with return value 0
Press any key to continue . . .

```

b) (1.0 điểm) Tìm số nào gần với số 300 nhất Trong các số nguyên tố vừa tìm được ở ý a)

Trả lời: Dán code bên dưới:

```
#include <iostream>
```

```
#include <math.h>
```

```
using namespace std;
```

```
bool checkPrime(int n)
```

```
{  
    if(n<2) return false;  
    for(int i=2; i<=sqrt(n); i++)  
    {  
        if(n%i==0) return false;  
    }  
    return true;  
}
```

```
void NearestPrime(int n)
```

```
{  
    int before, after;  
    before=n-1;  
    after=n+1;  
    while(!checkPrime(before) && !checkPrime(after))  
    {  
        before--;  
        after++;  
    }  
    if(checkPrime(before) && checkPrime(after)) cout <<before <<" " <<after;  
    else  
    {  
        if(checkPrime(before)) cout <<before;  
        else cout <<after;  
    }  
}
```

```

}

int main() {
    int n;
    cout << "Nhap gia tri n: ";
    cin >> n;

    cout << "So nguyen to gan "<<n<<" nhat la: ";
    NearestPrime(n);
    return 0;
}

```

Trả lời: Dán kết quả thực thi vào bên dưới:

```

Nhap gia tri n: 300
So nguyen to gan 300 nhat la: 293, 307
-----

```

Câu 2 (3 điểm): Cho ma trận A. Viết chương trình (có sử dụng hàm) phân rã ma trận A bằng phương pháp SVD.

Trả lời: Dán code vào bên dưới

```

#include <stdio.h>
#include <math.h>
#include <conio.h>
#include <iostream>
#include <iomanip>
#include <Eigen/Dense>
using Eigen::MatrixXd;
using namespace Eigen;
using namespace std;

void input(float a[][10], int r, int c)
{

```

```

        for(int i=0; i<r; i++)
        {
            for(int j=0; j<c; j++)
            {
                cout <<"a[" <<i <<"][" <<j <<"] = ";
                cin >>a[i][j];
            }
        }
    }

void output(float a[][10], int r, int c)
{
    for(int i=0; i<r; i++)
    {
        for(int j=0; j<c; j++)
        {
            cout << setw(12) << setprecision(5) << fixed <<a[i][j];
        }
        cout <<"\n";
    }
}

void saochep(float a[][10], float b[][10], int r, int c)
{
    for (int i=0;i<r; i++)
    {
        for (int j=0; j<c; j++)
        {
            b[i][j]=a[i][j];
        }
    }
}

```

```
void nhan(float a[][10], int r1, int c1, float b[][10], int r2, int c2, float t[][10])
```

```
{
    int i,j,k;
    if(c1!=r2)
    {
        cout <<"\nKhong the nhan 2 ma tran!\n";
        return;
    }
    for (i=0;i<r1;i++)
        for (j=0;j<c2;j++)
        {
            t[i][j]=0;
            for (k=0;k<c1;k++)
                t[i][j]+=a[i][k]*b[k][j];
        }
}
```

```
void chuyenvi(float a[][10], float aT[][10], int r, int c)
```

```
{
    for (int i=0;i<r; i++)
    {
        for (int j=0; j<c; j++)
        {
            aT[j][i]=a[i][j];
        }
    }
}
```

```
//Tim tri rieng _ Vector rieng
```

```
void Cal_Eival_Eivec(float s[][10], int r, int c, float lamda[10], float v[][10])
```

```
{
    MatrixXf Ae;
```

```

Ae.resize(r, c);
for (int i = 0; i < r; i++)
{
    for (int j = 0; j < c; j++)
    {
        Ae(i, j) = s[i][j];
    }
}
EigenSolver<MatrixXf> k(Ae);
VectorXcf kEigenvalue = k.eigenvalues();
for (int i = 0; i < kEigenvalue.size(); i++)
{
    lamda[i] = kEigenvalue(i).real();
}
MatrixXcf kEigenvector = k.eigenvectors();
for (int i = 0; i < r; i++)
{
    for (int j = 0; j < c; j++)
    {
        if (j%2==0)
        {
            v[i][j] = - kEigenvector(i, j).real();
        }
        else
            v[i][j] = kEigenvector(i, j).real();
    }
}
// Sap xep lai eigenValues, eigenVector
for (int i = 0; i < c - 1; i++)
{

```

```

    for (int j = i + 1; j < c; j++)
    {
        if (lamda[i] < lamda[j])
        {
            float temp = lamda[i];
            lamda[i] = lamda[j];
            lamda[j] = temp;
            for (int k = 0; k < c; k++)
            {
                float temp2 = v[k][i];
                v[k][i] = v[k][j];
                v[k][j] = temp2;
            }
        }
    }
}

void findSigmaMatrix(float lamda[10], float sigma[][10], int r, int c)
{
    for (int i=0; i<r; i++)
    {
        for (int j=0; j<c; j++)
        {
            sigma[i][j] = 0;
        }
    }

    for (int i=0; i<r; i++)
    {
        sigma[i][i] = sqrt(fabs(lamda[i]));
    }
}

```

```

}

void findUMatrix(float sigma[][10], float a[][10], float v[][10], float u[][10], int r, int c){
    for(int i=0; i<r; i++)
    {
        float ui[10][10];
        float sigmai = sigma[i][i];
        float vi[10][10];
        for(int j=0; j<c; j++)
        {
            vi[j][0] = v[j][i];
        }
        nhan(a,r,c,vi,c,1,ui);
        for(int j=0; j<r; j++)
        {
            ui[j][0] /= sigmai;
            u[j][i] = ui[j][0];
        }
    }
}

int main()
{
    float a[10][10], aT[10][10], s[10][10], lamda[10], v[10][10], sigma[10][10], vT[10][10],
    u[10][10], M1[10][10], M2[10][10];

    int r, c;

    cout <<"SINGULAR VALUE DECOMPOSITION\n";

    cout <<"\nNhap kich thuoc matran a: \n";

    cout <<"Nhap so hang: ";

    cin >>r;

    cout <<"Nhap so cot: ";

    cin >>c;

    cout <<"\nNhap matran A: \n";

```

```

input(a,r,c);
cout <<"\nMatran A:\n";
output(a,r,c);

chuyenvi(a,aT,r,c);
nhan(aT,c,r,a,r,c,s);
Cal_Eival_Eivec(s,c,c,lamda,v);
cout <<"\nTri rieng cua ma tran S: ";
for (int i=0; i<c; i++)
{
    cout <<lamda[i] <<" ";
}
cout <<endl;
cout <<"Vector rieng cua ma tran S:\n";
output(v,c,c);

findSigmaMatrix(lamda,sigma,r,c);
cout <<"\nMa tran Sigma:\n";
output(sigma,r,c);

chuyenvi(v,vT,c,c);
cout <<"\nMa tran v^T:\n";
output(vT,c,c);

findUMatrix(sigma,a,v,u,r,c);
cout <<"\nMa tran U:\n";
output(u,r,r);

nhan(u,r,r,sigma,r,c,M1);
nhan(M1,r,c,vT,c,c,M2);

```

```
cout<<"\nCheck: U.Sigma.V^T (= A)\n";  
output(M2,r,c);  
return 0;  
}
```

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 4 \\ 1 & 2 & 5 \end{bmatrix}, \text{ sai số } \varepsilon = 10^{-5}.$$

Trả lời: Dán kết quả thực thi vào bên dưới biết rằng



SINGULAR VALUE DECOMPOSITION

Nhap kich thuoc matran a:

Nhap so hang: 3

Nhap so cot: 3

Nhap matran A:

a[0][0] = 1

a[0][1] = 2

a[0][2] = 3

a[1][0] = 1

a[1][1] = 2

a[1][2] = 4

a[2][0] = 1

a[2][1] = 2

a[2][2] = 5

Matran A:

1.00000 2.00000 3.00000

1.00000 2.00000 4.00000

1.00000 2.00000 5.00000

Tri rieng cua ma tran S: 64.53513 0.46486 0.00001

Vector rieng cua ma tran S:

0.21301 -0.39323 0.89443

0.42602 -0.78645 -0.44722

0.87928 0.47630 0.00000

Ma tran Sigma:

8.03338 0.00000 0.00000

0.00000 0.68181 0.00000

0.00000 0.00000 0.00265

Ma tran v^T:

0.21301 0.42602 0.87928

-0.39323 -0.78645 0.47630

0.89443 -0.44722 0.00000

Ma tran U:

0.46094 -0.78795 -0.00085

0.57039 -0.08937 -0.00034

0.67985 0.60922 0.00017

Check: U.Sigma.V^T (= A)

1.00000 2.00000 3.00000

1.00000 2.00000 4.00000

1.00000 2.00000 5.00000

Câu 3 (2 điểm): Cho mười điểm trong không gian Oxy như sau: (2, 5); (3,7); (4,3); (2,9); (6,12); (7,16); (8,3); (9,8); (10,7); (11,12)

a) (1.0 điểm) Mô tả thuật toán xác định bao lồi

Trả lời: dàn sơ đồ khối hoặc mã giả

Sắp xếp các điểm: Sắp xếp tập hợp các điểm theo tọa độ x (nếu hai điểm có cùng tọa độ x, sắp xếp theo tọa độ y).

Tạo chuỗi dưới (Lower Hull): Duyệt qua tập hợp các điểm từ trái sang phải, thêm từng điểm vào chuỗi. Nếu việc thêm điểm mới tạo thành một góc không phải phải, loại bỏ điểm trước đó khỏi chuỗi cho đến khi góc phải được tạo thành. Lặp lại quá trình này cho đến khi tất cả các điểm đều được duyệt.

Tạo chuỗi trên (Upper Hull): Tương tự như bước 2, nhưng duyệt qua tập hợp các điểm từ phải sang trái.

Kết hợp hai chuỗi: Kết hợp chuỗi dưới và chuỗi trên để tạo thành bao lồi.

b) (1.0 điểm) Viết hàm xác định bao lồi và tính diện tích bao lồi tìm được.

Trả lời: Dán code bên dưới:

```
#include <bits/stdc++.h>

using namespace std;

#define MAX 100

struct Point
{
    int x, y;
    bool operator < (Point p)
    {
        return x < p.x || (x == p.x && y < p.y);
    }
    bool operator == (Point p)
    {
        return x == p.x && y == p.y;
    }
};

bool checkP(Point p[], int n, Point pn)
```

```

{
    int i=0;
    while(i<n)
    {
        if((p[i].x == pn.x) && (p[i].y == pn.y))
        {
            cout<<"Da ton tai diem nay: P["<<i<<"] = ("<<p[i].x<<","<<p[i].y<<")\n";
            cout<<"Vui long nhap lai diem khac\n";
            return 1;
        }
        i=i+1;
    }
    return 0;
}

void input(Point p[], int n)
{
    cout<<"Point[0].x = ";
    cin>>p[0].x;
    cout<<"Point[0].y = ";
    cin>>p[0].y;
    for(int i=1; i<n; i++)
    {
        do
        {
            cout<<"Point[" <<i <<"].x = ";
            cin>>p[i].x;
            cout<<"Point[" <<i <<"].y = ";
            cin>>p[i].y;
        } while(checkP(p, i, p[i]));
    }
}

```

```

    }

}

void output(Point p[], int n)
{
    for(int i=0; i<n; i++)
    {
        cout<<"(" <<p[i].x <<" , " <<p[i].y <<")\n";
    }
}

void copyMatrix(Point a[], Point b[], int n)
{
    for(int i=0; i<n; i++)
    {
        b[i].x = a[i].x;
        b[i].y = a[i].y;
    }
}

double cross_product(Point p, Point q, Point r)
{
    return (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
}

void sort(Point p[], int n)
{
    // selectionSort ascending x
    int i, j, min;
    for (i = 0; i < n - 1; i++)
    {
        min = i;
        for (j = i + 1; j < n; j++)
        {

```

```

        if (p[j] < p[min])
        {
            min = j;
        }
    }
    swap(p[min], p[i]);
}
}

// Tim bao loi
void convexHull_Mono(Point p[], int n, Point r[MAX], int *len)
{
    Point points[MAX];
    copyMatrix(p, points, n);
    if(n < 3)
    {
        cout<<"\nThere must be at least 4 points!";
        return;
    }
    sort(points, n);

    int k = 0;
    r[k++] = points[0];

    // Build upper hull
    for(int i=1; i<n; i++)
    {
        while((k >= 2) && (cross_product(r[k-2], r[k-1], points[i]) <= 0))
        {
            k--;
        }
    }
}

```

```

        r[k++] = points[i];
    }

    // Build lower hull
    for(int i=n-2, t=k+1; i>=0; i--)
    {
        while((k >= t) && (cross_product(r[k-2], r[k-1], points[i]) <= 0))
        {
            k--;
        }
        r[k++] = points[i];
    }
    k--;
    *len=k;
}

int orientation(Point p, Point q, Point r)
{
    // Compare slope of line segment => orientation
    int val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
    if (val == 0) return 0; // collinear
    return (val > 0)? 1: 2; // clock or counterclock wise
}

//Tinh dien tich bao loi
double polygonArea(Point p[], int n)
{
    double area = 0.0;
    int j = n - 1;
    for (int i = 0; i < n; i++)
    {
        area += (p[j].x + p[i].x) * (p[j].y - p[i].y);
    }
}

```

```

        j = i;
    }
    return abs(area / 2.0);
}

int main()
{
    Point p[MAX], r[MAX], d[MAX], pin[MAX], rpin[MAX];
    int n, len=0;
    cout<<"n = ";
    cin>>n;
    input(p,n);
    cout<<"-----\n";
    cout<<"\nYour input points:\n";
    output(p,n);
    cout<<"-----\n";
    cout<<"\nPoints of the convex hull MONOTUNECHAIN:\n";
    convexHull_Mono(p,n,r,&len);
    output(r,len);

    cout<<"-----\n";
    cout<<"Dien tich cua bao loi: ";
    cout<< polygonArea(r,len)<<endl;

    return 0;
}

```

4# Trả lời: Dán kết quả thực thi vào bên dưới:



D:\LEARNING\DaiHoc\HK5\Tc



```
n = 10
Point[0].x = 2
Point[0].y = 5
Point[1].x = 3
Point[1].y = 7
Point[2].x = 4
Point[2].y = 3
Point[3].x = 2
Point[3].y = 9
Point[4].x = 6
Point[4].y = 12
Point[5].x = 7
Point[5].y = 16
Point[6].x = 8
Point[6].y = 3
Point[7].x = 9
Point[7].y = 8
Point[8].x = 10
Point[8].y = 7
Point[9].x = 11
Point[9].y = 12
```

Your input points:

```
(2, 5)
(3, 7)
(4, 3)
(2, 9)
(6, 12)
(7, 16)
(8, 3)
(9, 8)
(10, 7)
(11, 12)
```

Points of the convex hull MONOTONECHAIN:

```
(2, 5)
(2, 9)
(7, 16)
(11, 12)
(10, 7)
(8, 3)
(4, 3)
```

Dien tích của bao lồi: 79

Câu 4 (2 điểm): Cho hàm số $f(x) = \ln(2x^2 + 1)^2 + 9x + 3e^{x^2} - 5$.

- a) (1 điểm) Trình bày thuật toán tối ưu hàm số đã cho sử dụng phương pháp *gradient descent* với *momentum*, biết rằng tham số học (learning rate) γ , hệ số động lượng là α

Trả lời: dàn sơ đồ khối hoặc mã giả

Input: Hàm số $f(x)$, điểm khởi đầu x_0 , hệ số động lượng (momentum coefficient) α , tham số học (learning rate) γ , số bước lặp N và sai số ε .

Output: giá trị bé nhất của $f(x)$ sử dụng phương pháp Gradient Descent với Momentum

Bước 1: Tính đạo hàm $f'(x)$

Bước 2: cho vòng lặp for(int i = 0; i <= step; i++)

Bước 2.1: double tmp = gamma * (gradient(x)) + alpha * (x - prev_x);

Bước 2.2: if (fabs(tmp) > esp) x -= tmp;

Bước 2.3: prev_x = x; }

Bước 3: Trả về giá trị nhỏ nhất của hàm $f(x)$ với x tương ứng.

- b) (1 điểm) Viết chương trình (có dùng hàm) tính giá trị bé nhất của $f(x)$ sử dụng phương pháp *gradient descent* với *momentum* với số bước lặp N và sai số ε :

Trả lời: Dán code vào bên dưới:

```
#include <iostream>
```

```
#include <cmath>
```

```
using namespace std;
```

```
double learningRate = 0.001; // Learning rate
```

```
double momentum = 0.5; // He so momentum
```

```
int maxIterations = 1000; // So lan lap toi da
```

```
double epsilon = 1e-5; // Sai so dung
```

```
// Gia tri ham so f(x)
```

```
double f(double x)
```

```
{
```

```
    return log(pow(2*x*x + 1, 2)) + 9*x + 3*exp(x*x) - 5;
```

```

}

// Đạo hàm bậc 1 của f(x)
double df(double x)
{
    return (8*x*(2*x*x + 1)) / (pow(2*x*x + 1, 2)) + 9 + 6*x*exp(x*x);
}

// Gradient Descent với Momentum
double gradientDescentWithMomentum(double initialX)
{
    double x = initialX;
    double velocity = 0.0;
    for (int i = 0; i < maxIterations; i++)
    {
        double gradient = df(x);
        velocity = momentum * velocity + (1.0 - momentum) * gradient;
        x -= learningRate * velocity;
        if (abs(gradient) < epsilon)
        {
            break;
        }
    }
    return x;
}

int main() {
    double x ; // Giá trị ban đầu của x
    cout << "Nhập giá trị x: " ;
    cin >> x;
}

```

```

cout <<"\n-----GRADIENT DESCENT WITH MOMENTUM-----\n";

double result = gradientDescentWithMomentum(x);

cout <<"Gia tri x toi uu: " <<result <<endl;

cout <<"Gia tri f(x) toi uu: " <<f(result) <<endl;


return 0;

}

```

Trả lời: **Dán kết quả thực thi** với điểm khởi $x = -1.00000$, tham số học học (*learning rate*) $\gamma = 0.001$, hệ số động lượng (*momentum coefficient*) là $\alpha = 0.5$, số bước lặp $N \geq 1000$ và sai số $\varepsilon = 10^{-5}$:

```

D:\LEARNING\DaiHoc\HK5\Tc
Nhap gia tri x: -1

-----GRADIENT DESCENT WITH MOMENTUM-----
Gia tri x toi uu: -0.663193
Gia tri f(x) toi uu: -5.04926

-----
Process exited after 5.847 seconds with return value 0
Press any key to continue . . . |

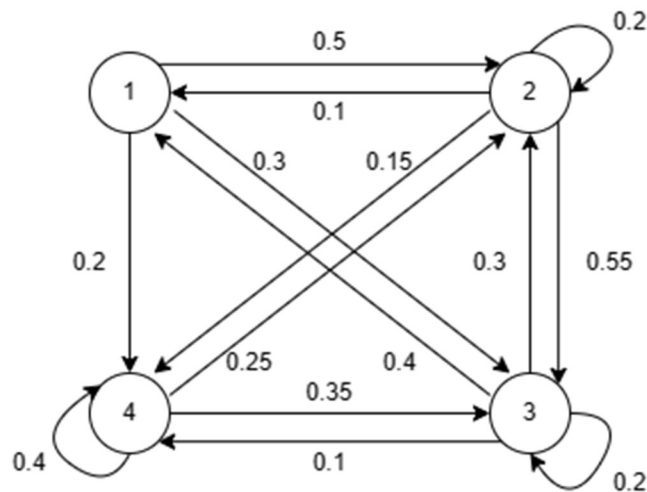
```

Câu 5 (1 điểm): Một hệ thống có chế độ làm việc ở mỗi giai đoạn vận hành chỉ với các trạng thái từ 1 đến 4. Chế độ làm việc của hệ thống này được mô tả bằng ma trận chuyển như sau:

$$P = \begin{bmatrix} 0 & 0.5 & 0.3 & 0.2 \\ 0.1 & 0.2 & 0.55 & 0.15 \\ 0.4 & 0.3 & 0.2 & 0.1 \\ 0 & 0.25 & 0.35 & 0.4 \end{bmatrix}$$

a) (0.5 điểm) Vẽ đồ thị **biểu** diễn chuỗi Markov tương ứng đã cho

Trả lời: **Dán kết quả vào bên dưới**



b) (0.5 điểm) Giả sử rằng hệ thống bắt đầu học ở trạng thái 3. Tính xác suất hệ thống làm việc ở trạng thái 1 sau 1, 2 và 3 bước thời gian vận hành.

Trả lời: Dán kết quả tính toán vào bên dưới:

Enter state to end: 1

Max probability in state 1, after 1 steps, start from state 3 is: 0.40000

Max probability in state 1, after 2 steps, start from state 3 is: 0.11000

Max probability in state 1, after 3 steps, start from state 3 is: 0.17850

GIẢNG VIÊN BIÊN SOẠN ĐỀ THI

Phạm Công Thắng

Đà Nẵng, ngày 04 tháng 12 năm 2023
KHOA CÔNG NGHỆ THÔNG TIN

(đã duyệt)

Nguyễn Văn Hiệu